

LLD Practice Platform — AI Collaboration & Decision Log

Architectural decisions contrasting AI suggestions against engineering rationale.

📅 Generated: September 2026 🏠 Project: LLD Practice Platform ⚡ Status: Verified & Production Ready

AI_USAGE.md — Engineering Decisions & AI Collaboration Log

This document records 4 meaningful architectural and technical decisions made during the design and implementation of the **LLD Practice Platform**, contrasting AI-assisted recommendations against final engineering trade-offs.

Decision 1: AI Evaluation Dependency & Graceful Fallback Architecture

- **Initial AI Suggestion:** Make the entire evaluation flow dependent on an external LLM API (such as OpenAI GPT-4o or Google Gemini), calling the LLM directly within the Django REST API view and returning the raw generated markdown string.
- **Accepted / Rejected:** REJECTED.
- **Engineering Rationale:**
 1. Relying exclusively on an external LLM introduces a hard failure mode whenever a learner has no API key configured, experiences network timeouts, or encounters API rate limits.
 2. Directly calling third-party APIs inside API views tightly couples the transport layer to the external vendor and makes automated unit testing difficult.
- **Final Implementation:**
 - Implemented the `SolutionEvaluator` abstraction with a deterministic `RuleBasedEvaluator` and a provider-agnostic `AIEvaluator`.
 - Created `CompositeEvaluator` which always runs deterministic static checks first, attempts AI qualitative reasoning, and gracefully falls back to deterministic results with `is_fallback=True` if the AI service fails.

Decision 2: Solution Input Modeling (Structured Entities vs Free-form Code / Canvas)

- **Initial AI Suggestion:** Provide a full-blown canvas with draggable UML class nodes (like React Flow) or a raw Java/C++ code editor with language server protocol (LSP) compilation.
- **Accepted / Rejected:** REJECTED.

- **Engineering Rationale:**
 1. A raw code editor forces the platform to parse entire programming language ASTs, introducing compiler-specific syntax errors that distract from object-oriented domain thinking.
 2. An interactive drag-and-drop canvas adds heavy front-end state synchronization complexity without guaranteeing structured semantic data for automated evaluation.
 - **Final Implementation:**
 - Designed a hybrid input model: structured class cards collecting class names, single responsibilities, typed methods, attributes, and relationships, combined with free-form architecture markdown, assumptions, and design pattern multi-selects.
 - Added a live Mermaid class diagram renderer with an automated "**Sync from Classes**" button, giving learners visual UML representation without fragile canvas state.
-

Decision 3: Deterministic vs AI Category Score Blending

- **Initial AI Suggestion:** Let the LLM output the final category scores (0–10) and overall score (0–100) directly, overriding all deterministic calculations.
 - **Accepted / Rejected:** REJECTED.
 - **Engineering Rationale:**
 1. LLMs exhibit non-deterministic variance and can be easily prompt-injected or hallucinate that a requirement is covered when no such class exists in the submission.
 2. Ground-truth structural metrics (e.g. presence of God classes with > 7 methods, dangling class relationship references, missing responsibilities, requirement keyword matches) are mathematically verifiable and should anchor the evaluation.
 - **Final Implementation:**
 - Implemented a weighted composite formula: **40% Deterministic Verification + 60% AI Qualitative Reasoning**.
 - In the UI, feedback items are explicitly tagged as either **Deterministic Check** (teal badge) or **AI Reasoning** (indigo badge) so learners clearly understand the provenance of each critique.
-

Decision 4: Attempt Immutability & Iterative Learning Revision Model

- **Initial AI Suggestion:** Allow learners to continuously edit the same attempt in-place, overwriting the previous evaluation results with each save.
- **Accepted / Rejected:** REJECTED.
- **Engineering Rationale:**
 1. Overwriting attempts destroys historical learning progression and prevents learners from comparing their initial naive design with their subsequent refactored architecture.
 2. In engineering interviews, iterative progression (identifying bottlenecks in Attempt #1 and fixing them in Attempt #2) is the primary learning signal.
- **Final Implementation:**

- Enforced attempt immutability at the domain service layer: once an attempt is in `SUBMITTED`, `EVALUATING`, or `COMPLETED` state, modifications to its classes or solution text are rejected with HTTP 400.
- Implemented the "Try Again" workflow which creates a new `PracticeAttempt` with `attempt_number = N + 1` for the same problem, paired with an **Attempt History & Progression Modal** to compare scores across iterations.